C++ PROGRAMMING

Complete Teaching Guide with Full Programs

Every program is complete and ready to copy, paste, and run

Table of Contents

1. Introduction to C++ 4
2. Structure of a C++ Program 5
3. Variables and Data Types 6
3.1 Type Modifiers (short, long, signed, unsigned) 6
4. Operators 8
4.1 Arithmetic Operators 8
4.2 Relational Operators (Comparison) 8
4.3 Logical Operators 9
4.4 Assignment & Increment/Decrement Operators 9
5. Input and Output 10
6. Type Casting 11
7. Storage Classes 12
8. Control Statements (Decision Making) 13
8.1 if / else if / else 13
8.2 switch Statement 13
9. Loops 14
9.1 for Loop — use when you know how many times to repeat 14
9.2 while Loop — use when repeat count is unknown 14
9.3 do-while Loop — runs at least once 14
9.4 break, continue 15
10. Functions 16
10.1 Function Overloading — same name, different parameters 16
10.2 Default Arguments 16
10.3 Recursion — a function calling itself 17
10.4 Pass by Value vs Pass by Reference 17
11. Arrays 19
11.1 2D Arrays (rows and columns) 19
12. Strings 20
13. Structures (struct) 21
14. Unions 22
15. Enumerations (enum) 23
16. Pointers and References 24
16.1 References — an easier alias for a variable 24
17. Dynamic Memory Allocation 25
18. Classes and Objects (OOP Basics) 26
19. Constructors and Destructors 27

20. Access Specifiers (Encapsulation) 28
21. Inheritance 29
22. Polymorphism 30
22.1 Compile-time Polymorphism (Function Overloading) 30
22.2 Run-time Polymorphism (Virtual Functions) 30
23. Exception Handling 32
24. Introduction to STL (Standard Template Library) 33
25. File Handling 34
26. Namespaces 35
27. Summary — Topics Covered 36

1. Introduction to C++

C++ is a programming language that lets you write fast programs and also organize code using real-world objects. It was created by Bjarne Stroustrup as an upgrade to the C language.

- Fast — compiles directly into machine code.
- Supports OOP — code can be organized as objects (Classes).
- Used in games, operating systems, browsers, banking software.

Real-world use: *Chrome browser, Windows OS, and most PC games are built using C++.*

2. Structure of a C++ Program

Every C++ program follows this same basic layout. Learn this structure first — everything else is built on top of it.

CODE

```
#include <iostream>       // brings in input/output tools (cin, cout)
using namespace std;      // lets us write cout instead of std::cout

int main() {              // program execution always starts here
    cout << "Hello World!" << endl;
    return 0;             // tells the system the program ended successfully
}
```

OUTPUT

```
Hello World!
```

EXPLANATION

Line 1 loads the iostream library so we can use cout and cin. Line 2 saves us from typing std:: before every command. int main() is the starting point of every program — the computer always begins here. cout << sends text to the screen, and endl moves to a new line. return 0; marks a successful finish.

- Every statement ends with a semicolon ;

⚠**Remember:** Forgetting the semicolon ; is the most common beginner mistake.

3. Variables and Data Types

A variable is a name given to a memory location that stores a value. Every variable must have a data type telling C++ what kind of value it holds.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int age = 20;
    float price = 99.5;
    char grade = 'A';
    bool isPass = true;
    string name = "Mohan";

    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Price: " << price << endl;
    cout << "Grade: " << grade << endl;
    cout << "Passed: " << isPass << endl;
    return 0;
}
```

OUTPUT

```
Name: Mohan
Age: 20
Price: 99.5
Grade: A
Passed: 1
```

EXPLANATION

Each variable is declared with its data type followed by a name and a value. Notice that isPass prints as 1, not 'true' — C++ displays bool values as 1 (true) or 0 (false) by default when printed with cout.

Data Type	Used For	Example
int	Whole numbers	int marks = 90;
float	Decimal numbers (small)	float pi = 3.14;
double	Decimal numbers (large/precise)	double gpa = 8.92;
char	A single character	char grade = 'A';
bool	true or false	bool isPass = true;
string	Text (words/sentences)	string name = "Mo";
void	No value (used for functions)	void greet();

3.1 Type Modifiers (short, long, signed, unsigned)

These change the size/range of a type. short and long change memory size; signed allows negative numbers, unsigned allows only zero and positive numbers.

CODE

```
#include <iostream>
```

```
using namespace std;

int main() {
    short int a = 100;
    long int b = 7000000;
    unsigned int c = 50;    // only 0 and positive values allowed

    cout << "short: " << a << endl;
    cout << "long: " << b << endl;
    cout << "unsigned: " << c << endl;
    return 0;
}
```

OUTPUT

```
short: 100
long: 7000000
unsigned: 50
```

4. Operators

4.1 Arithmetic Operators

CODE

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 3;
    cout << "Addition: " << a + b << endl;
    cout << "Subtraction: " << a - b << endl;
    cout << "Multiplication: " << a * b << endl;
    cout << "Division: " << a / b << endl;
    cout << "Modulus: " << a % b << endl;
    return 0;
}
```

OUTPUT

```
Addition: 8
Subtraction: 2
Multiplication: 15
Division: 1
Modulus: 2
```

EXPLANATION

Division of two integers (5 / 3) drops the decimal part, giving 1, not 1.67. Modulus (%) gives only the remainder of the division, which is 2 here.

> ⚠**Remember:** To get a decimal answer from division, make at least one number a float or double: 5.0 / 3.

4.2 Relational Operators (Comparison)

CODE

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 3;
    cout << "a == b: " << (a == b) << endl;
    cout << "a != b: " << (a != b) << endl;
    cout << "a > b: " << (a > b) << endl;
    cout << "a < b: " << (a < b) << endl;
    return 0;
}
```

OUTPUT

```
a == b: 0
a != b: 1
a > b: 1
a < b: 0
```

EXPLANATION

Relational operators always return a bool: 1 means true, 0 means false. These are the building blocks of every if condition and loop condition you will write.

4.3 Logical Operators

CODE

```
#include <iostream>
using namespace std;

int main() {
    bool hasUsername = true;
    bool hasPassword = false;

    cout << "AND: " << (hasUsername && hasPassword) << endl;
    cout << "OR: " << (hasUsername || hasPassword) << endl;
    cout << "NOT: " << (!hasUsername) << endl;
    return 0;
}
```

OUTPUT

```
AND: 0
OR: 1
NOT: 0
```

Real-world use: *Login systems check username && password — both must be true before allowing access.*

4.4 Assignment & Increment/Decrement Operators

CODE

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    a += 5;   cout << "After +=5: " << a << endl;
    a -= 3;   cout << "After -=3: " << a << endl;
    a++;      cout << "After a++: " << a << endl;
    a--;      cout << "After a--: " << a << endl;
    return 0;
}
```

OUTPUT

```
After +=5: 15
After -=3: 12
After a++: 13
After a--: 12
```

5. Input and Output

cin takes input from the keyboard. cout displays output on the screen.

CODE

```
#include <iostream>
using namespace std;

int main() {
    string name;
    int age;

    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your age: ";
    cin >> age;

    cout << "Hello " << name << ", you are " << age << " years old." << endl;
    return 0;
}
```

OUTPUT

```
Enter your name: Mohan
Enter your age: 22
Hello Mohan, you are 22 years old.
```

EXPLANATION

cin >> reads whatever the user types and stores it in the given variable. Multiple cin/cout statements can be used one after another to collect several inputs in sequence.

> ⚠**Remember:** cin >> stops at a space. Use getline(cin, name) to read a full sentence with spaces.

6. Type Casting

Type casting means converting one data type into another.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 3;
    double result = (double)a / b;   // converts a to double before dividing
    cout << "Without casting: " << a / b << endl;
    cout << "With casting: " << result << endl;
    return 0;
}
```

OUTPUT

```
Without casting: 3
With casting: 3.33333
```

EXPLANATION

(double)a forces 'a' to be treated as a decimal number before the division happens, so the result keeps its decimal part instead of being truncated.

- Implicit casting: C++ converts automatically (int to double).
- Explicit casting: You force the conversion using (type).

Real-world use: *Splitting a total bill among people needs casting to get a decimal answer, not a rounded-down whole number.*

7. Storage Classes

Storage classes control where a variable lives and how long it stays in memory.

Keyword	Meaning
auto	Default class; local to the block (most common)
static	Keeps its value between function calls
extern	Refers to a variable defined in another file
register	Suggests storing the variable in fast CPU memory

CODE

```
#include <iostream>
using namespace std;

void counter() {
    static int count = 0;   // remembers value between calls
    count++;
    cout << "Count: " << count << endl;
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

OUTPUT

```
Count: 1
Count: 2
Count: 3
```

EXPLANATION

Normally a local variable resets every time a function runs. static keeps count's value alive between calls instead of resetting it to 0 each time — that's why it keeps increasing.

8. Control Statements (Decision Making)

8.1 if / else if / else

CODE

```
#include <iostream>
using namespace std;

int main() {
    int marks = 85;
    if (marks >= 90) {
        cout << "Grade A" << endl;
    } else if (marks >= 75) {
        cout << "Grade B" << endl;
    } else {
        cout << "Grade C" << endl;
    }
    return 0;
}
```

OUTPUT

```
Grade B
```

EXPLANATION

Conditions are checked top to bottom. Since marks (85) is not >= 90 but is >= 75, the second block runs and the rest is skipped.

8.2 switch Statement

Used when one variable is compared against many fixed values.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int day = 3;
    switch (day) {
        case 1: cout << "Monday"; break;
        case 2: cout << "Tuesday"; break;
        case 3: cout << "Wednesday"; break;
        default: cout << "Invalid";
    }
    return 0;
}
```

OUTPUT

```
Wednesday
```

⚠**Remember:** Never forget 'break;' — without it, execution falls through into the next case.

9. Loops

9.1 for Loop — use when you know how many times to repeat

CODE

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";
    }
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
```

9.2 while Loop — use when repeat count is unknown

CODE

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    while (i <= 5) {
        cout << i << " ";
        i++;
    }
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
```

9.3 do-while Loop — runs at least once

CODE

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    do {
        cout << i << " ";
        i++;
    } while (i <= 5);
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
```

Real-world use: *A 'Play Again? (y/n)' prompt uses do-while since the game must run once before asking.*

9.4 break, continue

CODE

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i == 5) break;        // stops the loop entirely
        if (i % 2 == 0) continue;  // skips even numbers
        cout << i << " ";
    }
    return 0;
}
```

OUTPUT

```
1 3
```

EXPLANATION

The loop stops completely once i reaches 5 (break). Before that, even numbers (2 and 4) are skipped by continue, so only 1 and 3 get printed.

10. Functions

A function is a reusable block of code that performs one task. It helps break a big program into smaller, manageable parts.

CODE

```
#include <iostream>
using namespace std;

int add(int a, int b) {   // function definition
    return a + b;
}

int main() {
    int result = add(5, 3);   // function call
    cout << "Sum: " << result << endl;
    return 0;
}
```

OUTPUT

```
Sum: 8
```

EXPLANATION

add(int a, int b) takes two numbers and returns their sum. main() calls it with 5 and 3, and stores the returned value in result before printing it.

10.1 Function Overloading — same name, different parameters

CODE

```
#include <iostream>
using namespace std;

int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }

int main() {
    cout << "Int add: " << add(2, 3) << endl;
    cout << "Double add: " << add(2.5, 3.1) << endl;
    return 0;
}
```

OUTPUT

```
Int add: 5
Double add: 5.6
```

EXPLANATION

Both functions are named add, but C++ picks the right one automatically based on whether you pass whole numbers or decimals.

10.2 Default Arguments

CODE

```
#include <iostream>
using namespace std;
```

```
double area(double l, double w = 1.0) {
    return l * w;
}

int main() {
    cout << "area(5): " << area(5) << endl;
    cout << "area(5,3): " << area(5, 3) << endl;
    return 0;
}
```

OUTPUT

```
area(5): 5
area(5,3): 15
```

10.3 Recursion — a function calling itself

CODE

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n <= 1) return 1;        // base case — stops recursion
    return n * factorial(n - 1);
}

int main() {
    cout << "5! = " << factorial(5) << endl;
    return 0;
}
```

OUTPUT

```
5! = 120
```

EXPLANATION

factorial(5) calls factorial(4), which calls factorial(3), and so on until n reaches 1 (the base case). The results then multiply back up: 5×4×3×2×1 = 120.

> ⚠**Remember:** Every recursive function needs a base case, or it will call itself forever and crash.

10.4 Pass by Value vs Pass by Reference

CODE

```
#include <iostream>
using namespace std;

void addTen(int &num) {   // & means reference — changes the original
    num += 10;
}

int main() {
    int x = 5;
    addTen(x);
    cout << "x after addTen: " << x << endl;
```

```
    return 0;
}
```

OUTPUT

```
x after addTen: 15
```

EXPLANATION

Because num is declared with &, it refers directly to x's memory — so changes made inside the function affect the original variable, not just a copy.

11. Arrays

An array stores multiple values of the same type under a single name, accessed using an index starting from 0.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {90, 85, 78, 92, 88};

    cout << "First: " << marks[0] << endl;
    cout << "All marks: ";
    for (int i = 0; i < 5; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT

```
First: 90
All marks: 90 85 78 92 88
```

⚠**Remember:** marks[5] does not exist in a 5-element array — valid indexes are 0 to 4.

11.1 2D Arrays (rows and columns)

CODE

```
#include <iostream>
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

OUTPUT

```
1 2 3
4 5 6
```

Real-world use: *2D arrays are used to represent spreadsheets, chessboards, and images.*

12. Strings

A string is a sequence of characters used to store text. C++ provides an easy-to-use string type.

CODE

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name = "Mohanraj";
    cout << "Name: " << name << endl;
    cout << "Length: " << name.length() << endl;
    cout << "First 4 letters: " << name.substr(0, 4) << endl;
    string full = "Hello " + name;
    cout << full << endl;
    return 0;
}
```

OUTPUT

```
Name: Mohanraj
Length: 8
First 4 letters: Moha
Hello Mohanraj
```

EXPLANATION

.length() counts the characters. .substr(0, 4) extracts 4 characters starting from position 0. The + operator joins two strings together.

13. Structures (struct)

A structure groups different data types together under one name — useful for representing a single real-world entity.

CODE

```
#include <iostream>
using namespace std;

struct Student {
    string name;
    int marks;
};

int main() {
    Student s1;
    s1.name = "Mohan";
    s1.marks = 90;
    cout << s1.name << " scored " << s1.marks << endl;
    return 0;
}
```

OUTPUT

```
Mohan scored 90
```

EXPLANATION

Student is a blueprint holding a name and marks together. s1 is a variable of type Student, and the dot (.) operator accesses its fields.

Real-world use: *A struct can represent one student's record, one employee's data, or one point (x, y) on a graph.*

14. Unions

A union is like a structure, but all its members share the same memory location — only one member can hold a value at a time.

CODE

```
#include <iostream>
using namespace std;

union Data {
    int i;
    float f;
};

int main() {
    Data d;
    d.i = 10;
    cout << "As int: " << d.i << endl;
    d.f = 5.5;              // this overwrites the same memory used by d.i
    cout << "As float: " << d.f << endl;
    return 0;
}
```

OUTPUT

```
As int: 10
As float: 5.5
```

⚠**Remember:** Unions save memory but are used less often than structures — mainly in low-level/embedded programming.

15. Enumerations (enum)

An enum lets you create a variable that can only hold one of a fixed set of named values — makes code more readable than using plain numbers.

CODE

```
#include <iostream>
using namespace std;

enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY };

int main() {
    Day today = WEDNESDAY;
    if (today == WEDNESDAY) {
        cout << "Midweek!" << endl;
    }
    cout << "Value: " << today << endl;
    return 0;
}
```

OUTPUT

```
Midweek!
Value: 2
```

EXPLANATION

Behind the scenes, enum values are just numbers starting from 0 (MONDAY=0, TUESDAY=1, WEDNESDAY=2...), but writing WEDNESDAY in code is far more readable than writing 2.

Real-world use: *Traffic light states (RED, YELLOW, GREEN) are commonly modeled as an enum.*

16. Pointers and References

A pointer stores the memory address of another variable, instead of storing the value directly.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int age = 25;
    int *ptr = &age;    // ptr stores the address of age

    cout << "Value of age: " << age << endl;
    cout << "Address of age: " << &age << endl;
    cout << "Value via pointer: " << *ptr << endl;
    return 0;
}
```

OUTPUT

```
Value of age: 25
Address of age: 0x7ffd3a1b2c4c
Value via pointer: 25
```

EXPLANATION

&age gets the memory address where age lives. *ptr means 'go to that address and get the value stored there' — this is called dereferencing. The exact address printed will differ each time you run it.

⚠**Remember:** An uninitialized pointer can crash a program. Always assign it an address, or set it to nullptr.

16.1 References — an easier alias for a variable

CODE

```
#include <iostream>
using namespace std;

int main() {
    int age = 25;
    int &ref = age;   // ref is another name for age
    ref = 30;
    cout << "age is now: " << age << endl;
    return 0;
}
```

OUTPUT

```
age is now: 30
```

17. Dynamic Memory Allocation

Normally memory is fixed at compile time. 'new' and 'delete' let a program create and free memory while running.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int *p = new int;    // allocate memory for one int
    *p = 50;
    cout << "Value: " << *p << endl;
    delete p;             // free the memory when done

    int *arr = new int[5];   // allocate an array dynamically
    for (int i = 0; i < 5; i++) arr[i] = i * 10;
    for (int i = 0; i < 5; i++) cout << arr[i] << " ";
    cout << endl;
    delete[] arr;             // free array memory
    return 0;
}
```

OUTPUT

```
Value: 50
0 10 20 30 40
```

⚠**Remember:** Forgetting 'delete' causes a memory leak — memory stays reserved even after it's no longer needed.

18. Classes and Objects (OOP Basics)

A class is a blueprint. An object is a real item built from that blueprint, with its own data.

CODE

```
#include <iostream>
using namespace std;

class Student {
    public:
        string name;
        int marks;
        void display() {
            cout << name << " scored " << marks << endl;
        }
};

int main() {
    Student s1;             // object created from class
    s1.name = "Mohan";
    s1.marks = 92;
    s1.display();
    return 0;
}
```

OUTPUT

```
Mohan scored 92
```

EXPLANATION

Student is the class (blueprint) with attributes name/marks and a method display(). s1 is an actual object created from that blueprint, and the dot operator calls its method.

Real-world use: *A 'Patient' class in a hospital app bundles name, age, diagnosis together with actions like admitPatient().*

19. Constructors and Destructors

A constructor runs automatically when an object is created — used to set up initial values. A destructor runs automatically when the object is destroyed — used to clean up.

CODE

```
#include <iostream>
using namespace std;

class Student {
    public:
        string name;
        Student(string n) {       // constructor - same name as class
            name = n;
            cout << "Object created for " << name << endl;
        }
        ~Student() {               // destructor - starts with ~
            cout << "Object destroyed: " << name << endl;
        }
};

int main() {
    Student s1("Mohan");    // constructor runs automatically
    cout << "Program running..." << endl;
    return 0;                // destructor runs automatically here
}
```

OUTPUT

```
Object created for Mohan
Program running...
Object destroyed: Mohan
```

EXPLANATION

The constructor Student(string n) fires the moment s1 is created, setting up its name. The destructor ~Student() fires automatically when the program ends and s1 goes out of scope.

20. Access Specifiers (Encapsulation)

Access specifiers control which parts of a program can access a class's data — this protection is called encapsulation.

Specifier	Access
public	Accessible from anywhere
private	Accessible only inside the class (default for class)
protected	Accessible inside the class and its child classes

CODE

```
#include <iostream>
using namespace std;

class Account {
    private:
        double balance;    // hidden from outside – protected data
    public:
        Account() { balance = 0; }
        void deposit(double amt) { balance += amt; }
        double getBalance() { return balance; }
};

int main() {
    Account a1;
    a1.deposit(500);
    a1.deposit(250);
    cout << "Balance: " << a1.getBalance() << endl;
    return 0;
}
```

OUTPUT

```
Balance: 750
```

EXPLANATION

balance cannot be accessed directly from main() (a1.balance would give an error) because it's private. It can only be changed safely through deposit() and read through getBalance().

Real-world use: *A bank account's balance is kept private so it can only be changed through safe functions like deposit(), not directly.*

21. Inheritance

Inheritance lets a new class (child) reuse the data and functions of an existing class (parent), and add its own.

CODE

```
#include <iostream>
using namespace std;

class Animal {              // parent class
    public:
        void eat() { cout << "This animal eats food" << endl; }
};

class Dog : public Animal {  // child class inherits from Animal
    public:
        void bark() { cout << "The dog barks" << endl; }
};

int main() {
    Dog d;
    d.eat();    // inherited from Animal
    d.bark();   // Dog's own function
    return 0;
}
```

OUTPUT

```
This animal eats food
The dog barks
```

EXPLANATION

Dog automatically gets eat() from Animal through inheritance (: public Animal), without rewriting it, and also defines its own new function bark().

Type	Meaning
Single	One child inherits from one parent
Multiple	One child inherits from more than one parent
Multilevel	A child class becomes parent for another class
Hierarchical	Multiple children inherit from one parent

Real-world use: *A 'SavingsAccount' class can inherit common features from a general 'Account' class.*

22. Polymorphism

Polymorphism means 'many forms' — the same function name behaves differently depending on how it is used.

22.1 Compile-time Polymorphism (Function Overloading)

CODE

```
#include <iostream>
using namespace std;

int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }

int main() {
    cout << add(2, 3) << endl;
    cout << add(2.5, 1.5) << endl;
    return 0;
}
```

OUTPUT

```
5
4
```

22.2 Run-time Polymorphism (Virtual Functions)

CODE

```
#include <iostream>
using namespace std;

class Animal {
    public:
        virtual void sound() { cout << "Animal sound" << endl; }
};

class Dog : public Animal {
    public:
        void sound() override { cout << "Bark" << endl; }
};

int main() {
    Animal *a;      // pointer of parent type
    Dog d;
    a = &d;
    a->sound();     // calls Dog's version, not Animal's
    return 0;
}
```

OUTPUT

```
Bark
```

EXPLANATION

virtual tells C++ to decide at runtime which version of sound() to run, based on the actual object type (Dog), even though 'a' is declared as an Animal pointer.

23. Exception Handling

Exception handling lets a program deal with runtime errors gracefully instead of crashing.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 0;
    try {
        if (b == 0) throw "Cannot divide by zero";
        cout << a / b;
    } catch (const char* msg) {
        cout << "Error: " << msg << endl;
    }
    return 0;
}
```

OUTPUT

```
Error: Cannot divide by zero
```

- try: the code that might cause an error.
- throw: signals that an error occurred.
- catch: handles the error so the program keeps running.

Real-world use: *Banking apps use exception handling so a failed transaction shows an error message instead of crashing the app.*

24. Introduction to STL (Standard Template Library)

The STL provides ready-made data structures and algorithms so you don't have to build them from scratch.

CODE

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> marks = {90, 85, 78};   // a resizable array
    marks.push_back(95);                  // add an item
    for (int m : marks) {
        cout << m << " ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT

```
90 85 78 95
```

EXPLANATION

Unlike a normal array, a vector can grow in size. push_back() adds a new value at the end, and the for (int m : marks) loop reads every element in order.

Container	Use
vector	A resizable array
map	Stores key-value pairs
set	Stores unique values, automatically sorted
stack	Last-In-First-Out storage
queue	First-In-First-Out storage

25. File Handling

File handling lets a program read from and write to files, so data isn't lost when the program closes.

CODE

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("data.txt");   // open file for writing
    outFile << "Hello File!";
    outFile.close();

    ifstream inFile("data.txt");    // open file for reading
    string line;
    getline(inFile, line);
    cout << "Read from file: " << line << endl;
    inFile.close();
    return 0;
}
```

OUTPUT

```
Read from file: Hello File!
```

EXPLANATION

ofstream creates/opens a file for writing. ifstream opens a file for reading. Always close() a file after you're done with it.

Real-world use: *A student result system saves marks to a file so they're still available the next time the program runs.*

26. Namespaces

A namespace groups related code under one name to avoid naming conflicts between different parts of a program.

CODE

```
#include <iostream>
using namespace std;

namespace School {
    int studentCount = 100;
}

int main() {
    cout << "Students: " << School::studentCount << endl;
    return 0;
}
```

OUTPUT

```
Students: 100
```

⚠**Remember:** 'using namespace std;' is why we can write cout instead of std::cout — std is the Standard Library's namespace.

27. Summary — Topics Covered

- Introduction, program structure, data types, operators, input/output
- Type casting, storage classes
- Control statements (if, switch) and loops (for, while, do-while)
- Functions: overloading, default arguments, recursion, pass by reference
- Arrays, strings, structures, unions, enumerations
- Pointers, references, dynamic memory allocation
- OOP: classes, objects, constructors, destructors, access specifiers
- Inheritance and polymorphism
- Exception handling, STL basics, file handling, namespaces

Every program in this guide is complete and compiles on its own — copy, paste, and run directly to demonstrate live in class.